

The Boundary of Graph Sparsification for Community Detection

Mohammad Dindoost, Bavan DivaaniAazar, Ioannis Koutis, and David A. Bader

Abstract—

Index Terms—Graph sparsification, community detection, modularity, evaluation methodology, graph reduction.

I. Introduction

Community detection is a fundamental task in network analysis [1], [2], and a computationally demanding one. The most widely used methods, including modularity optimization [3], [4], [5] and flow-based approaches [6], have running times that scale with the number of edges, and on the graphs to which they are now routinely applied that quantity is large [7].

Graph sparsification constructs a subgraph on the same vertex set with substantially fewer edges while preserving properties of the original. The strongest guarantees are spectral. Spielman and Srivastava [8] show that sampling each edge with probability proportional to its effective resistance, and reweighting the retained edges by the inverse of that probability, yields a subgraph whose Laplacian quadratic form approximates that of the original within a factor $1 \pm \epsilon$ [9], [10]. Spectral approximation is a statement about the Laplacian, and the eigenvectors of the Laplacian encode cuts and connectivity [11], [12]. Community structure is closely related to those quantities without being identical to them. Because effective resistances are themselves expensive to compute, practical variants approximate them. DSpar [13] replaces the resistance of an edge (u, v) by $1/d_u + 1/d_v$, which bounds it within a factor $2/\alpha$ where α is the normalized spectral gap [14], and thereby obtains a spectral sparsifier at the cost of reading the degree sequence. If the properties preserved by sparsification include community structure, then detection can be performed on the sparsified graph at lower cost and without loss of quality. Whether they do is the question this paper addresses. The answer we obtain is conditional. On the networks and detection algorithms examined here, two conditions have to hold together: the algorithm must take the number of communities as an input, and the graph must carry edges whose removal does not destroy its community structure. Where either condition fails we find no gain on the objective, no end-to-end speed benefit, and no gain in recovery that survives comparison with an unsparsified run of a better detector.

Satuluri et al. [15] introduced local sparsification, L-Spar, in which each vertex retains the incident edges whose endpoints have the highest Jaccard similarity of neighbourhoods. They reported speedups commonly between $10\times$ and $50\times$ with little or no loss of cluster quality, and improved accuracy for two of the four clustering algorithms they tested.

The approach was taken up and extended. Hamann et al. [16] conducted a systematic comparison of structure-preserving sparsification methods for social networks and released reference implementations, which are now distributed in standard graph libraries. Wu and Chen [17] trained a generative model to sparsify graphs specifically for community detection, and report that clustering accuracy is retained while as little as five percent of the edges are kept. Sparsification also entered production: the SimClusters system at Twitter [18] prunes its interaction graph before community detection at industrial scale. Most recently, Chen et al. [19] benchmarked twelve sparsification algorithms against sixteen graph properties on fourteen networks, concluding that no single sparsifier preserves all properties and that the choice should be matched to the downstream task.

The conditions under which the original result was obtained are specific, and none of them travelled with it. The accuracy gain was reported for fixed- k cut partitioners, judged against external ground-truth communities, on dense graphs, and against clustering runs that took hours. The pipeline is now applied to modularity optimizers that choose their own granularity, judged partly by objectives computed on the graph itself, on sparse graphs, and against detection algorithms that run in near-linear time. The same paper reports a synthetic sweep in which the method begins to outperform clustering on the unsparsified graph at an average degree of approximately 50, and to our knowledge no subsequent work examines that threshold.

These extensions also change what has to be measured. The question a sparsify-then-detect pipeline is meant to answer is whether the communities of the original graph can still be recovered after most of its edges are removed. The object of interest is therefore the original graph, and the quantity to be compared is the quality, on that graph, of two partitions: the one obtained by running the detection algorithm on the sparsified graph, and the one obtained by running it on the original graph directly. Both partitions cover the same vertex set, so both can be scored on the same object, and the difference between those two

scores is the effect of sparsification.

The sparsifier classes and detection algorithms described above each impose further requirements of the same kind, and several have been noted individually in the literature. Similarity-based sparsifiers [15] remove between-community edges preferentially, and those are the edges that modularity and conductance penalize, so the sparsifier and the quality measure act on the same quantity. Degree-based sparsifiers [13] select on endpoint degrees, and a heavy-tailed degree sequence reproduces several of the resulting effects with no community structure present, so a degree-preserving random graph [20] is needed to separate the two. Detection algorithms that choose their own granularity [4], [5] return finer partitions on a sparsified graph: Chen et al. [19] document community counts rising by three orders of magnitude across a pruning sweep, and Hamann et al. [16] warn that normalized mutual information is inflated when sparsification fragments the graph, recommending chance-corrected indices in its place. Finally, sparsification is itself a computation, and Gottesbüren et al. [21] observe that its overhead can counteract the speedup it is intended to produce. Each requirement is a property of a specific method class rather than a general caveat, and each bounds what a comparison involving that class can conclude. Section III states the requirements and the control that establishes each one.

Applying these requirements, we re-examined the question across seven sparsifiers, spanning the degree-based and similarity-based families that have claimed quality gains, the method of Satuluri et al. among them, and a connectivity-preserving class that cannot fragment a graph by construction; four detection algorithms drawn from the two families identified above; seventeen real networks of up to 25 million edges, seven of which carry the full quality sweep and all seventeen the mechanism measurements; and two families of synthetic benchmark, LFR graphs and stochastic block models, in which average degree is swept over an order of magnitude while community structure is held fixed. One arm of the synthetic study uses a fixed- k partitioner, so that the number of communities is identical in the sparsified and unsparsified conditions and granularity is excluded by construction rather than by matching. The study comprises 23 controlled experiments. Predictions and stopping rules were fixed before each experiment was run, and several of our own hypotheses did not survive them.

Of the two conditions, the second has a location. Average degree is the correlate we can vary under control, and measured against it the transition between the two regimes is sharp within a given family of graphs while its location is not universal. On the LFR benchmark graphs [22] we generate it falls near an average degree of 50, which is also the threshold reported in the original synthetic sweep. On the real networks we test it has already occurred by an average degree of 29 and is absent on every network below 11, so it lies between those two densities. On stochastic block models it occurs no later than on LFR. We therefore state a condition rather than

a threshold.

What the transition tracks is how many edges can be removed without destroying community structure, rather than density as such. Injecting spurious edges into a graph makes sparsification more effective, which is the signature that quantity predicts, and average degree is its strongest measured correlate. Degree heterogeneity is not a correlate at all. Plain and degree-corrected block models generated at the same average degree and mixing cross at the same place although their degree variability differs by a factor of ten, and across those graphs the gain correlates with average degree at Spearman $+0.65$ and with degree variability at -0.26 . The measurement noise present in how a graph was collected remains a correlate we have not separated.

Where either condition fails, the three ways a benefit could appear all close. For detection algorithms that choose their own granularity, on the sparse real networks we test, no sparsifier improves modularity measured on the original graph beyond run-to-run variation, at any retention that preserves quality, and this holds for every such algorithm we tested. There is no speed benefit either. Measured end to end, including the cost of the sparsifier, the pipeline is no faster than clustering the original graph directly, and at aggressive retention it is slower, because a fragmented graph presents the optimizer with more work rather than less. Every apparent gain we found in this regime dissolves when granularity is matched, when chance is subtracted, or when the baseline is given an equal compute budget, with one exception on one network, where the surviving recovery gain is instead exceeded by an unsparsified run of a different detection algorithm.

Where both conditions hold, the gain is real. On synthetic graphs with planted communities, with the number of communities fixed by construction so that no granularity effect is possible, similarity-based sparsification raises both the objective measured on the original graph and chance-corrected agreement with the planted labels. Recovery turns positive at a lower density than the objective does. On the benchmark graphs, degree-based and uniform sparsification at identical retention do not reproduce the effect, which places it in the selection rule rather than in the edge budget; on the one real network above the transition, a degree-based sampler reproduces it as well, while the two methods that preserve connectivity by construction do not. What the gain requires is a rule that scores individual edges, and we do not identify the property of that rule which matters.

A fixed- k partitioner applied to a sparse graph shows no gain on the objective in any configuration we tested, and a free-granularity optimizer applied to a dense one shows none either once granularity is controlled. Neither condition is sufficient on its own. Which sparsifier is used matters less than either condition, provided its selection signal reflects local structure rather than degree alone. The mechanism is consistent with this: what governs a degree-based sparsifier's effect on modularity is where high-degree-product edges sit relative to community bound-

aries, which we establish by direct intervention.

Two qualifications attach to this result. Sparsification repairs a weaker detector rather than surpassing a stronger one. On the synthetic benchmarks, a resolution-tuned modularity optimizer on the untouched graph outperforms every sparsified pipeline we measured, and the same holds on all but one of the real networks we test. And with an exact similarity computation the quality gain is not a speed gain, because the sparsifier costs more than the detection it accelerates.

This paper contributes the following.

- The boundary. The conditions under which sparsification helps, namely a detection algorithm that takes the number of communities as an input and a graph carrying sufficient removable redundancy, established under controls that exclude granularity by construction, together with the negative territory delimited across seven sparsifiers and four detection algorithms on seven real networks. The negative result is not an artifact of the fragmentation that sparsification induces: a connectivity-preserving sparsifier that cannot disconnect a graph leaves no vertex in a small component, and modularity falls by the same margin. We also report that these methods have a hard retention floor on sparse graphs, so that aggressive sparsification is not merely unhelpful there but unreachable.
- An evaluation protocol, and the measured cost of omitting it. Each control is motivated by a specific failure it catches, and the consequence of the central omission is quantified at 45 sign reversals in 63 configurations. We also introduce a permuted-weight control for similarity-weighted pipelines, which shows that the benefit attributed to similarity weighting is reproduced, and in one case exceeded, when the weights are randomly permuted across edges.
- A mechanism for the boundary. A decomposition of the score-separation quantity into a sorting term and a granularity term across seventeen networks, and a direct causal test in which that quantity is steered through more than a full sign change while degree sequence, partition, intra-edge fraction and modularity are all held exactly fixed.
- An open phenomenon. In several controlled settings the partitions that best recover reference communities are not the partitions that score best on the objective, and the two criteria move in opposite directions within the same configuration. We report this as an observation rather than a result, and argue that it is the most consequential open question the boundary exposes.

II. Background and Setting

III. Evaluation Protocol

A sparsify-then-detect pipeline is a claim about a graph that no longer exists in the pipeline’s output. That is the source of every difficulty in evaluating it, and each control

below follows from it. We state the controls here and apply them without further comment in Section IV, where their consequences are measured.

A. One yardstick for two partitions

Let $\mathcal{A}$ be a detection algorithm, G the graph whose communities are wanted, and G' a sparsified copy of it. The pipeline returns $P_{\text{sparse}} = \mathcal{A}(G')$ and the alternative to it is $P_{\text{base}} = \mathcal{A}(G)$. Both are partitions of the same vertex set, and the question the pipeline exists to answer is which of them better describes G . Both are therefore scored on G :

$$\Delta Q_{\text{transfer}} = Q(P_{\text{sparse}}, G) - Q(P_{\text{base}}, G), \quad (1)$$

$$\Delta Q_{\text{self}} = Q(P_{\text{sparse}}, G') - Q(P_{\text{base}}, G). \quad (2)$$

The second accounting also appears in this literature. It differs from the first in one place, the graph on which the first term is evaluated, and that difference is not a detail of bookkeeping. Equation (1) measures two partitions with one instrument. Equation (2) changes the instrument between the arm under test and the arm it is tested against, and gives the arm under test an instrument built by deleting edges from the other one.

The requirement is not ours. It is stated, and followed, in the paper that introduced the pipeline [15, §4.3].

Why (2) favours the treatment follows from the objective. Modularity scores a partition P of a graph with m edges by

$$Q(P, G) = \sum_{C \in P} \left[\frac{m_C}{m} - \gamma \left(\frac{D_C}{2m} \right)^2 \right], \quad (3)$$

with m_C the number of edges inside community C , D_C the sum of degrees in C , and γ a resolution parameter equal to one in the standard definition. Removing edges does not by itself move this quantity for a fixed partition: the intra-community fraction and the subtracted term fall together. What changes is the value an optimizer can reach. A graph with fewer edges admits higher modularity, and does so even when it has no community structure at all [23], so a partition optimized on G' can score well on G' while describing G worse than the baseline does. Equation (2) reads that difference as an improvement.

B. Matching granularity

Sparsification fragments a graph, and a detection algorithm that chooses its own number of communities responds by returning more of them. Agreement measures are not neutral to that. Purity is non-decreasing under refinement. Normalized mutual information rises with the number of clusters, which Hamann et al. [16] identified as making it unsuitable for sparsified networks, pointing to the adjusted Rand index instead [24]. Average best-match F_1 over reference communities rises for the same reason: with more clusters, some cluster is more likely to match a given reference community well. A comparison

between partitions of different granularity therefore measures granularity along with whatever else it is meant to measure.

The control is to give the baseline the same freedom. We partition the original graph at a matched community count by raising γ in (3) until the counts agree, and compare the sparsified partition against that rather than against the optimizer’s default. If a finer partition of the untouched graph recovers the reference communities as well, the sparsifier has contributed nothing that the resolution parameter does not.

Where the counts cannot be brought together we draw no recovery conclusion. The comparability rule is $|\Delta k|/k < 25\%$, and the cells it excludes are reported as excluded.

C. Subtracting chance

A partition that contains no information at all does not score zero against reference communities, and what it does score depends on how many parts it has. Chance-corrected indices remove this by construction, which is what adjustment means, and we use them wherever the reference communities form a partition.

Where they do not, and the reference communities overlap and are numerous, we use average best-match F_1 , which has no such correction. For every arm we therefore generate a random partition with the same community size distribution as the partition being scored, run the same metric on it, and report the result beside the measurement. The floor moves with the same quantity the previous control governs, so the two are applied together and never as substitutes.

D. An equal compute budget

Every detection algorithm used here is stochastic, so one run is one sample from a distribution over partitions, and a pipeline that sparsifies before detecting has spent more time than one run of the detector. Comparing a pipeline against a single baseline run credits the pipeline with the difference in effort.

The registered control gives the baseline as many independent restarts as fit the pipeline’s wall clock and takes the best of them. This control has a weakness that runs in the treatment’s favour: when the pipeline is fast, the budget it buys is small, and a small number of restarts may fail to sample a good mode of a multi-modal algorithm. The baseline then looks worse than the algorithm is, and the pipeline inherits the difference.

We therefore report two baselines. The compute-matched one is the fair accounting of what the pipeline costs. The best of a fixed number of restarts is a strictly more generous baseline, which can only handicap the treatment and never manufacture a gain. A result counts as surviving only if it survives both. Since the controls compose rather than substitute, the granularity control is applied to results that have already passed this one.

E. Nulls

Two quantities that this literature reads as evidence about community structure can be produced without any community structure present. Each needs a null that detects the substitution.

a) A degree sequence without communities: A degree-based sparsifier selects on a function of the degree sequence alone, so any effect it produces on a heavy-tailed graph may follow from that sequence and not from the communities in it. The control is a degree-preserving rewiring [20], generated by double edge swaps, which holds every vertex degree exactly while destroying the community structure. A signal that reappears on the rewiring is not evidence about communities, whatever its size on the real graph.

b) Weights without their placement: A related family of proposals deletes no edges and instead reweights each one by a similarity score. The corresponding null permutes the computed weights uniformly at random across the edges. This preserves the weight distribution exactly and destroys every association between a weight and the structure that produced it, so a gain that survives the permutation is a gain from weight heterogeneity and not from similarity. We are not aware of this control being used in this literature.

F. Realized retention

Retention is the measured fraction of edges kept, and the parameter a sparsifier is given does not determine it. Sampling schemes that clip per-edge probabilities at one retain fewer edges than their nominal parameter implies, and the shortfall grows with the skew of the score distribution. Local selection rules that give every vertex a quota cannot go below the retention that quota implies, whatever target they are given, which puts a hard floor under them on sparse graphs. Comparing two sparsifiers at equal nominal parameters therefore compares them at unequal amounts of deletion. We compare at matched realized retention and report the realized value everywhere.

A comparison must also be like for like in what is removed. A method that deletes vertices as well as edges scores its partition on a smaller and denser subpopulation, which is an easier problem, so the fraction of vertices retained belongs beside the fraction of edges.

G. Cost measured end to end

The sparsifier is a computation and belongs to the pipeline that uses it. The comparison is the whole pipeline, sparsifier included, against one run of the same detector on the original graph. Charging the sparsification to nobody can turn a loss into a saving on paper, and it was charged correctly in the original report [15]; Gottesbüren et al. [21] observe the same overhead in graph partitioning.

TABLE I

The protocol. Each control answers a specific way in which a comparison between a sparsified pipeline and an unsparsified baseline can favour the pipeline without any improvement having occurred. Section IV measures what each one is worth.

How the comparison can mislead	Control
Quality read off the sparsified graph, which admits higher modularity	Score both partitions on the original graph, Eq. (1)
Finer partitions score higher on agreement measures	Partition the original graph at a matched community count
Uncorrected measures pay for nothing, and pay more as parts multiply	Chance-corrected indices, and a size-matched random-partition floor
The pipeline has spent more compute than the baseline	Compute-matched restarts and the best of a fixed number
A degree-derived score acts through the degree sequence	Degree-preserving rewiring
A weighting is credited to similarity	Permute the weights across edges
Nominal and realized retention differ, and differ by method	Compare at measured retention; report floors and vertex coverage
Sparsification is charged to nobody	Time the whole pipeline, sparsifier included

IV. Experiments

A. What was tested, and why these methods

Seven sparsifiers were run through the protocol of Section III, chosen so that a negative result cannot be attributed to the selection.

Two are the methods whose quality claims motivate the pipeline: local sparsification, L-Spar [15], and the degree-based sampler DSpar [13], one from each family described in Section II. L-Spar and the method NetworKit calls Local Similarity are distinct and both appear below; L-Spar always refers to the method of Satuluri et al. Three more are the methods that the largest independent benchmark of sparsification ranks highest at preserving clustering structure, namely K-Neighbor, Local Degree and Local Similarity [19], [16], so that the arms include the strongest candidates chosen by someone else. The remaining two are a connectivity-preserving backbone, a spanning structure topped up to the retention target either at random or by similarity, which cannot disconnect a graph by construction and is included as an instrument rather than as a competitor. Uniform random deletion supplies the no-information baseline wherever a selection rule has to be shown to be doing work.

Two classes of sparsifier are excluded, and the exclusions bound what follows. Node-removing methods such as k -core peeling are not tested, because they score their partition on a smaller and denser vertex set, which is a different comparison requiring the coverage control of Section III-F. Learned sparsifiers [17] are not tested,

TABLE II

Hard retention floors by method, as a fraction of edges. A blank marks a method with no floor. Networks are ordered by average degree; the floor is governed by it. Below these values the method cannot be run at all, whatever the target.

Network	d_{avg}	Backbone	L-Spar	Loc. Deg.	K-Neigh.
com-Amazon	5.53	0.362	0.301	0.352	0.330
ca-HepTh	5.74	0.348	0.276	0.347	0.317
com-DBLP	6.62	0.302	0.244	0.301	0.277
ca-CondMat	8.55	0.234	0.186	0.234	0.218
email-Enron	10.73	0.186	0.159	0.186	0.179
wiki-Vote	28.51	0.070	0.067	0.070	0.069
email-Eu-core	32.58	0.061	0.053	0.061	0.060

because matching them on realized retention requires retraining at each operating point.

Two detection algorithms are also absent, and the reason is not a design choice. The founding accuracy claim concerns Metis, Graclus and MLR-MCL. Metis is tested here, on every network, through the pymetis bindings. For Graclus and MLR-MCL we could find no maintained implementation that we were able to run, so the algorithm whose behaviour the original claim is largest about, MLR-MCL, remains untested by us. That is the most consequential gap in this paper and we return to it in Section VII.

Before any quality measurement, one structural fact constrains the design. Four of the seven methods cannot reach aggressive retention on a sparse graph, because each vertex keeps at least one incident edge. Table II gives the floors. On the three sparsest networks no method in this study can retain less than about a third of the edges, so the aggressive regime in which sparsification would pay for itself is not merely unhelpful there; it is unreachable. All arms are therefore compared at a common operating point per network, and the third point is set at the largest floor rather than at a round number.

B. The objective

Table III reports the result. Across 102 matched-retention cells, seven sparsifiers on seven networks under a modularity optimizer, not one cell improves modularity measured on the original graph, whether the baseline is the compute-matched control or the best of five plain restarts. The best cell in the entire grid loses 0.0105.

The sweep that isolates one method across its full retention range agrees. Local L-Spar on the same seven networks loses between 0.014 and 0.404 against a compute-matched baseline in every configuration that runs, with the loss growing as retention falls, and its three most aggressive configurations cannot be run at all because of the floors in Table II.

C. Four ways the loss could be an artifact

A negative result is only as good as the explanations it excludes. Four are available, and each was tested rather than argued.

TABLE III

Every matched-retention cell, Leiden, seven networks. ΔQ is measured on the original graph, Eq. (1), against the best of five plain restarts. Counts are cells with a positive value. The arms differ in how much they lose and not in whether they lose.

Arm	Cells	$\Delta Q > 0$	Best cell	Worst cell
K-Neighbor	18	0	-0.0105	-0.667
DSpar	18	0	-0.0116	-0.640
L-Spar	9	0	-0.0117	-0.407
Backbone, random fill	14	0	-0.0141	-0.214
Local Similarity	15	0	-0.0172	-0.311
Local Degree	14	0	-0.0288	-0.278
Backbone, Jaccard fill	14	0	-0.0290	-0.235
Total	102	0	-0.0105	-0.667

a) Fragmentation: Sparsification breaks graphs apart, and the optimizer may be penalized for being handed a shattered graph. The backbone arms test this directly, because they retain a spanning structure and cannot increase the number of connected components. They behave as designed: under the backbone with random fill no detected community is a singleton in any of its fourteen cells, and the fraction of vertices in communities of fewer than three members never exceeds 1.8%, where the shattering arms reach 86% on the same networks. The conclusion does not move. The backbone’s best cell is -0.0141, worse than three of the arms that fragment freely. Removing half the edges costs modularity on the original graph whether or not the graph stays in one piece.

b) The choice of optimizer: Everything above uses a modularity optimizer, so the objective and the algorithm may be too closely coupled. Two sparsifiers across three further algorithms, one flow-based, one a local majority rule and a second modularity optimizer, give the 63 configurations of Table ?? . Against the best of N plain restarts, 60 are negative, with mean changes of -0.082 under Infomap, -0.046 under Louvain and -0.106 under label propagation. The three positive cells are label propagation on one network whose unsparified baseline is pathological: its own best of twenty restarts reaches modularity 0.079 where a modularity optimizer on the same graph reaches 0.416. Relieving a known failure mode of one algorithm is not a benefit of sparsification.

c) A benign loss: The loss might be harmless if sparsification sheds peripheral vertices while leaving community cores intact. It does not. The fragments subdivide communities without mixing them, but they cut the most embedded vertices as readily as the least, and a resolution-matched partition of the original graph preserves cores better in sixteen of twenty comparisons. Restricting the damage to leaf and near-leaf vertices does not rescue it either.

d) The retention regime: The loss might be an artifact of pushing the methods too hard. It is not: it is already present at the mildest retention any of them can reach. On the three sparsest networks that is about a third of the edges, by Table II, and every arm loses there.

The regime in which sparsification would pay for itself is not merely unprofitable on these graphs, it is unreachable.

One observation from the same arms does not belong to this list, because it supports no rival explanation. The backbone’s two fills differ only in which edges top up the spanning structure, random or highest Jaccard similarity, and random fill scores better in ten of the fourteen paired cells. In this regime the similarity signal subtracts.

D. The same result under a fixed- k partitioner

Sections IV-B and IV-C concern detectors that choose their own granularity. The condition under which sparsification does help, established in this paper on synthetic benchmarks, involves a detector that takes the number of communities as an input, so the question is whether such a detector escapes the negative on these graphs.

It does not. Metis was run on all seven networks at the same operating points and under the same protocol, with ten partitioner seeds per cell and the worst sparsified run compared against the best baseline run. On the five networks with average degree between 5.5 and 10.7, no cell is positive: 90 cells, seven sparsifiers, two ways of choosing k , and the best of them loses 0.017. Fixing the community count removes the granularity artifact by construction and leaves the loss exactly where it was.

The two conventions for choosing k agree throughout, whether k is taken from the community count a modularity optimizer returns on the original graph or, on the three networks with reference communities, from the number of those communities. On com-Amazon the two values differ by a factor of sixteen, 306 against 5,000, and both give the same answer.

Above that range the picture changes, and Section VII takes it up: on the densest network we test, four of the seven sparsifiers produce a gain that survives the worst case over ten seeds.

E. Recovery, and what happens to it under control

Agreement with reference communities behaves differently from the objective, and the difference is instructive rather than encouraging.

Raw agreement often rises, and it rises because the sparsified partition is finer. Table ?? shows the anatomy on com-DBLP: an apparent gain of +0.091 in average best-match F_1 , against a control that gains +0.198 without touching the graph, over a chance floor that itself rises from 0.019 to 0.090 as the partition fragments. Both controls are needed to see it, and after both there is nothing left.

This is the pattern across the labelled networks, with one exception, which is the subject of Section IV-G.

F. No speed benefit either

The pipeline is not faster. Of the 63 configurations, four preserve quality in the sense of coming within 0.005 of the best-of- N baseline, and the fastest of those four

runs at $0.66\times$ the speed of clustering the original graph directly. Twelve configurations exceed $1.5\times$, and every one of them loses modularity, by 0.015 to 0.162. Timing only the detection step and giving the sparsifier away for free, the median speedup is $1.21\times$, $1.63\times$ and $1.15\times$ by algorithm, well short of the factor of two that halving the edge count suggests, because all three algorithms return more communities on the sparsified graph and do more work per pass.

G. The exception

One result in this section survives every control we have.

On com-Amazon, L-Spar at realized retention 0.542 reaches average best-match F_1 of 0.402 against a baseline of 0.142. The granularity control does not remove it: partitioning the original graph to the same community count gives 0.319, and deliberately over-matching by 37 per cent more communities still gives only 0.372. The chance floor does not remove it either, and subtracting the floor widens the margin rather than narrowing it, because the floor rises with the community count. A second, independent instance appears in the same experiments from an unrelated method, Local Degree, on com-Amazon and com-DBLP.

What removes it is a different comparison. Both non-modularity algorithms reach higher agreement on com-Amazon without any sparsification at all, 0.465 under Infomap and 0.480 under label propagation, against the 0.402 that the sparsified modularity pipeline attains. The effect is real, and it is the modularity objective's default granularity being partially repaired by shattering the graph, in a metric that rewards fine partitions. It is not a capability that sparsification confers.

Two things about this cell are worth carrying forward. It coexists with a modularity loss of 0.079 in the same configuration, so the objective and the recovery measure move in opposite directions on the same partition of the same graph. And it is the reason the negative result of this section is stated as it is: no benefit on the objective anywhere, and no recovery benefit that survives comparison against a better algorithm run on the untouched graph. Section VII takes up the dissociation as an open phenomenon.

V. The Mechanism

VI. Related Work

VII. Discussion

VIII. Conclusion

References

- [1] S. Fortunato, "Community detection in graphs," *Physics Reports*, vol. 486, no. 3–5, pp. 75–174, 2010.
- [2] S. Fortunato and M. E. Newman, "20 years of network community detection," *Nature Physics*, vol. 18, no. 8, pp. 848–850, 2022.
- [3] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Physical Review E*, vol. 69, no. 2, p. 026113, 2004.
- [4] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.
- [5] V. A. Traag, L. Waltman, and N. J. Van Eck, "From Louvain to Leiden: Guaranteeing well-connected communities," *Scientific Reports*, vol. 9, no. 1, pp. 1–12, 2019.
- [6] M. Rosvall and C. T. Bergstrom, "Maps of random walks on complex networks reveal community structure," *Proceedings of the national academy of sciences*, vol. 105, no. 4, pp. 1118–1123, 2008.
- [7] J. Leskovec, K. J. Lang, A. Dasgupta, and M. W. Mahoney, "Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters," *Internet Mathematics*, vol. 6, no. 1, pp. 29–123, 2009.
- [8] D. A. Spielman and N. Srivastava, "Graph sparsification by effective resistances," in *Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing*, 2008, pp. 563–568.
- [9] A. A. Benczúr and D. R. Karger, "Randomized approximation schemes for cuts and flows in capacitated graphs," *SIAM Journal on Computing*, vol. 44, no. 2, pp. 290–319, 2015.
- [10] D. A. Spielman and S.-H. Teng, "Nearly linear time algorithms for preconditioning and solving symmetric, diagonally dominant linear systems," *SIAM Journal on Matrix Analysis and Applications*, vol. 35, no. 3, pp. 835–885, 2014.
- [11] U. von Luxburg, "A tutorial on spectral clustering," *Statistics and Computing*, vol. 17, no. 4, pp. 395–416, 2007.
- [12] J. Shi and J. Malik, "Normalized cuts and image segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 8, pp. 888–905, 2000.
- [13] Z. Liu, K. Zhou, Z. Jiang, L. Li, R. Chen, S.-H. Choi, and X. Hu, "Dspar: An embarrassingly simple strategy for efficient GNN training and inference via degree-based sparsification," *Transactions on Machine Learning Research*, 2023.
- [14] L. Lovász, "Random walks on graphs," *Combinatorics, Paul Erdős Is Eighty*, vol. 2, no. 1–46, p. 4, 1993.
- [15] V. Satuluri, S. Parthasarathy, and Y. Ruan, "Local graph sparsification for scalable clustering," in *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data*, 2011, pp. 721–732.
- [16] M. Hamann, G. Lindner, H. Meyerhenke, C. L. Staudt, and D. Wagner, "Structure-preserving sparsification methods for social networks," *Social Network Analysis and Mining*, vol. 6, no. 1, p. 22, 2016.
- [17] H.-Y. Wu and Y.-L. Chen, "Graph sparsification with generative adversarial network," in *2020 IEEE International Conference on Data Mining (ICDM)*, 2020.
- [18] V. Satuluri, Y. Wu, X. Zheng, Y. Qian, B. Wichers, Q. Dai, G. M. Tang, J. Jiang, and J. Lin, "SimClusters: Community-based representations for heterogeneous recommendations at Twitter," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2020, pp. 3183–3193.
- [19] Y. Chen, H. Ye, S. Vedula, A. Bronstein, R. Dreslinski, T. Mudge, and N. Talati, "Demystifying graph sparsification algorithms in graph properties preservation," *Proceedings of the VLDB Endowment*, vol. 17, no. 3, pp. 427–440, 2024.
- [20] M. Molloy and B. Reed, "A critical point for random graphs with a given degree sequence," *Random Structures & Algorithms*, vol. 6, no. 2–3, pp. 161–180, 1995.
- [21] L. Göttesbüren, N. Maas, D. Rosch, P. Sanders, and D. Seemaier, "Linear-time multilevel graph partitioning via edge sparsification," in *33rd Annual European Symposium on Algorithms (ESA)*, 2025, arXiv:2504.17615.
- [22] A. Lancichinetti, S. Fortunato, and F. Radicchi, "Benchmark graphs for testing community detection algorithms," *Physical Review E*, vol. 78, no. 4, p. 046110, 2008.
- [23] R. Guimerà, M. Sales-Pardo, and L. A. N. Amaral, "Modularity from fluctuations in random graphs and complex networks," *Physical Review E*, vol. 70, no. 2, p. 025101, 2004.
- [24] N. X. Vinh, J. Epps, and J. Bailey, "Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance," *Journal of Machine Learning Research*, vol. 11, pp. 2837–2854, 2010.